

# Deep Dive Into Snapenhance Web (Analysis and Testing)

---

## Introduction

---

The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.

Masking is defined as:

- Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
- Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
- Making an account look relatively active (snap score not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
- Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence

All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

## Context

---

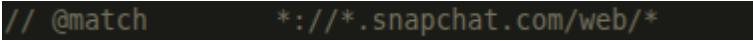
The purpose of this document is to analyze how Snapenhance currently works as-is in a web browser. The reason for this, it to see exactly what can and cannot be done as of yet. Furthermore, the features promised (if

relevant to the scope of the project) will be tested on the dummy accounts; one logged in on Google Chrome and one on a Samsung Galaxy S20.

## Code analysis

---

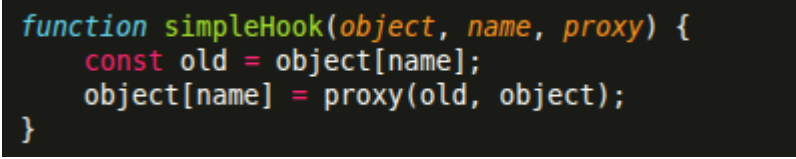
Included here will only be the relevant pieces of code from the script.

1. 

*match.png*

Allows the script to operate on any URL that matches the mentioned format.

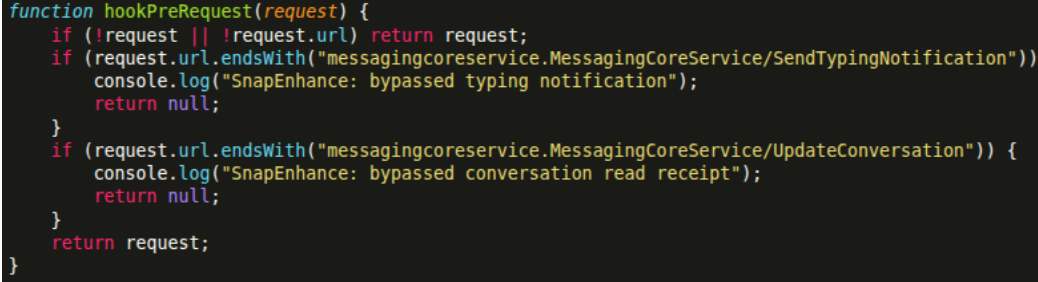
2. simpleHook



*proxy.png*

Takes an object from the original website and stores it in a constant named `old`, then replaces that object with a new version created by the proxy function, which can add extra features or change how the tool works while still using the original object saved in `old` to keep things running smoothly.

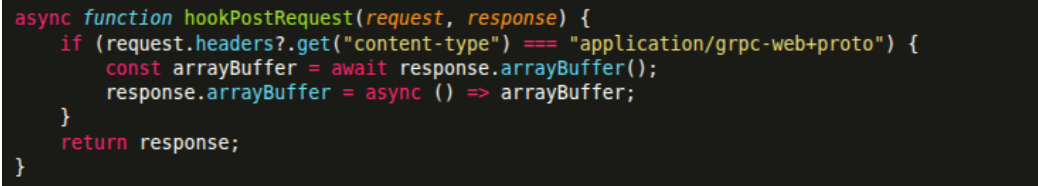
3. hookPreRequest



*hookPreRequest.png*

Looks at a request going out to the server and checks if it contains a message being typed or a message being read. If it does contain the mentioned, it trashes the request, not sending it out.

4. hookPostRequest



*hookPostRequest.png*

Checks replies from the server, and if they use Snapchat's format `application/grpc-web+proto`, it reads the data once and saves it in an `ArrayBuffer`. This is because Snapchat messages get removed after you have read them, so like this they are saved.

## 5. WebSocket

```
WebSocket.prototype.send = new Proxy(WebSocket.prototype.send, {
  apply: function (target, thisArg, argumentsList) {
    console.log("SnapEnhance WebSocket send:", argumentsList[0]);
    return target.apply(thisArg, argumentsList);
  }
});
```

*WebSocket.png*

The Websocket function yields no result at the moment, but sets up a functionality we can later elaborate on, allowing us to intercept and potentially modify real-time messages sent by Snapchat's website, such as those controlling Bitmoji avatars or live chat updates. For now, it simply logs what's being sent to the console, acting like a spy who listens in on the conversation without changing it. In the future, we could extend this to block specific messages (like hiding Bitmoji), alter their content, or add new features, enhancing the script's ability to customize the Snapchat Web experience while maintaining privacy and control.

```
const oldFetch = fetch;
fetch = async (...args) => {
  args[0] = hookPreRequest(args[0]);
  if (args[0] == null) {
    return Promise.resolve(new Response(null, { status: 200 }));
  }

  const requestBody = args[0].body;
  if (requestBody && !requestBody.locked && requestBody.getReader) {
    const buffer = await requestBody.getReader().read();
    args[0] = new Request(args[0], {
      body: buffer.value,
      headers: args[0].headers
    });
  }

  const result = oldFetch(...args);
  return new Promise(async (resolve, reject) => {
    try {
      resolve(await hookPostRequest(args[0], await result));
    } catch (e) {
      console.info("SnapEnhance fetch error:", e);
      reject(e);
    }
  });
};
```

6.

*oldFetch.png*

Utilizes `hookPreRequest` and `hookPostRequest`. Saves the original fetched data into a variable called `oldFetch`. The new `fetch` function checks every outgoing request using `hookPreRequest` (to block things like typing or

read receipt notifications), handles special data like streams (data that disappears after being read), and uses `hookPostRequest` to check replies from the server. If a request is blocked, it sends a fake “success” response to trick Snapchat into thinking everything worked. This ensures that privacy features (like hiding typing or read receipts) work in the background workers, where Snapchat handles a lot of chat-related tasks. By copying streams and checking replies, it keeps the website running smoothly without errors.

```
7. const oldBlobClass = window.Blob;
   window.Blob = class HookedBlob extends Blob {
     constructor(...args) {
       const data = args[0][0];
       if (typeof data === "string" && data.startsWith("importScripts")) {
         args[0][0] += `${workerInjected.toString()};workerInjected();`;
         window.Blob = oldBlobClass; // restore original Blob after injection
       }
       super(...args);
     }
   };
```

*blob.png*

A blob is a container that holds any kind of data. In this case, it holds a script that Snapchat uses to create a worker. The code here saves the old blob and make a new one, `HookedBlob`, that builds on the original. It adds `workerInjected()` to the old blob, which finally allows for private chat reading and typing (as `hookPreRequest` and `hookPostRequest` are wrapped in this function).

```
8. simpleHook(document, "createElement", (proxy, instance) => (...args) => {
   const result = proxy.call(instance, ...args);
   if (["audio", "video", "img"].includes(args[0])) {
     simpleHook(result, "setAttribute", (proxy2, instance2) => (...args2) => {
       result.style = "pointer-events: auto;";
       if (args2[0] === "controlsList") return;
       proxy2.call(instance2, ...args2);
     });
     result.addEventListener("load", () => {
       result.parentElement?.addEventListener("contextmenu", (event) => {
         event.stopImmediatePropagation();
       });
     });
     result.addEventListener("contextmenu", (event) => {
       event.stopImmediatePropagation();
     });
   }
   return result;
});
```

*simpleHook.png*

Uses the `simpleHook` function mentioned earlier and creates a modified overlay of the Snapchat Website, making images and videos clickable/saveable, with `document.createElement` and `pointer-events: auto`. It blocks the `controlsList` attribute which Snapchat uses to block video and image downloads.

```
9. simpleHook(document, "hasFocus", () => () => true);
```

*hasFocus.png*

Makes Snapchat think the website is always in focus, even when going on a different page.

```
10. const oldAddEventListener = EventTarget.prototype.addEventListener;
    Object.defineProperty(EventTarget.prototype, "addEventListener", {
      value: function (...args) {
        const eventName = args[0];
        if (eventName === "keydown" || eventName === "focus") return;
        return oldAddEventListener.call(this, ...args);
      }
    });
```

*oldEventListener.png*

Saves original event listeners and replaces them with events that don't trigger with certain keys pressed or focus change.

## Promised deliverables

---

Based on these snippets I should be able to:

1. Type messages without someone seeing I am typing
2. Save images and videos
3. Read messages without the read receipt being sent
4. Prevent Snapchat from tracking keyboard inputs or focus changes

## Testing

---

**Environment Browser + Snapenhance:** Chrome browser (version 138.0.7204.183) on Linux Mint. Extensions: Tampermonkey

**Environment Phone:** Physical Android Phone (model: Galaxy S20 Ultra 5G). OS: Android 13. App Version: official Snapchat (13.62.0.54).

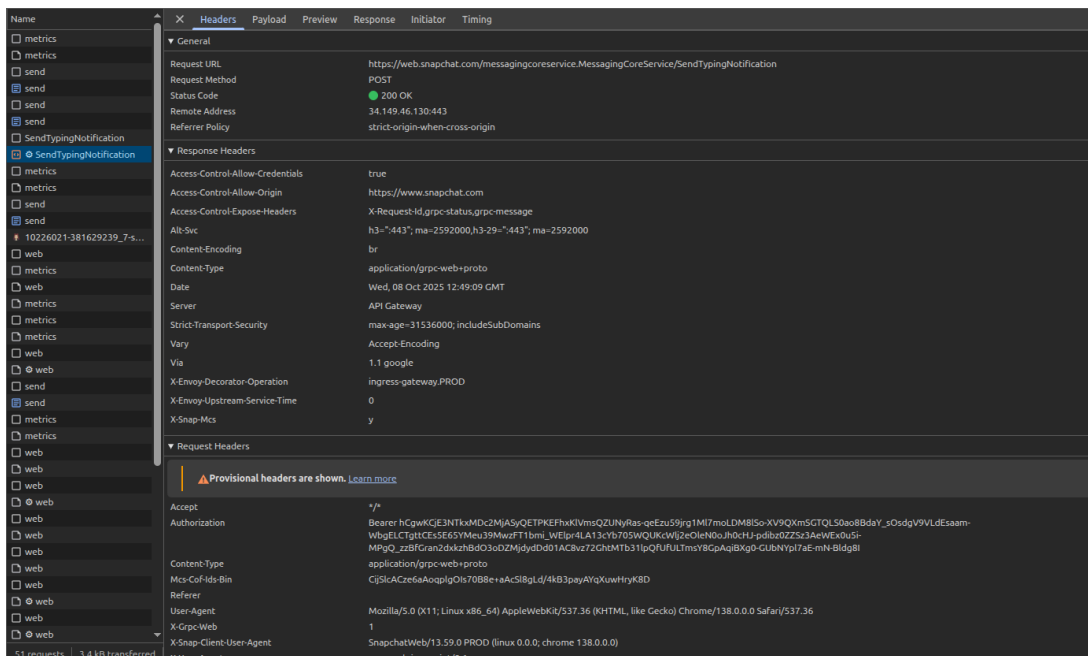
### Test 1 - Type without being seen typing

#### Expected Result:

The script is supposed to stop Snapchat from telling others when you're typing a message. It does this by blocking a specific web request (called SendTypingNotification) that Snapchat sends when you type.

**Result:** In both cases I could see that I was typing. This was not masked. The code however specified hiding your bitmoji status, and in the case of the web account, no bitmoji was made. The test will be redone with a bitmoji on the web account as well.

**Result 2:** I added a Bitmoji to the web account. Typing was still visible on both devices, so the script failed to hide it.



requestURL.png

Console error:

```
da337bca022b7d53abce.chunk.js:2 SnapEnhance: bypassed typing notification
0272cff8546fe5a3413b.js:2 Uncaught (in promise)
MessagingError: sendTypingNotification error: INTERNALERROR
    at Proxy.onError (0272cff8546fe5a3413b.js:2:4888338)
    at MessagePort.n (0272cff8546fe5a3413b.js:2:5296689)
    at MessagePort.r (0272cff8546fe5a3413b.js:2:3831883)
onError @ 0272cff8546fe5a3413b.js:2
n @ 0272cff8546fe5a3413b.js:2
r @ 0272cff8546fe5a3413b.js:2
```

### Why It Didn't Work:

- **Request Interception Succeeded but Caused an Error:** The console shows “SnapEnhance: bypassed typing notification,” meaning the script correctly detected and blocked the `SendTypingNotification` request to `https://web.snapchat.com/messagingcoreservice.MessagingCoreService/SendTypingNotification`. However, the `INTERNALERROR` suggests Snapchat’s website noticed the request was blocked and threw an error, possibly still sending the typing notification another way (e.g., via WebSockets or a fallback mechanism).
- **Possible WebSocket Issue:** The script only blocks fetch requests, but Snapchat might send typing info through WebSockets.

- **Worker Injection Problem:** The script adds its blocking code to Snapchat's background workers (hidden programs handling chats). The error suggests the worker might be crashing or behaving unexpectedly when the script interferes.

## Test 2 - Download images and videos

**Expected Result:** The script is supposed to make images and videos downloadable by right-clicking them on Snapchat's website. It does this by making media clickable with pointer-events: auto and blocking the `controlsList` attribute that prevents downloads, while also bypassing screenshot detection notifications.

**Result:** Saving images worked. I could right-click and download images on Chrome without Snapchat sending a screenshot alert to the phone.

**Result 2:** Videos didn't display properly, just a black screen with audio. This happened with and without the script, so I couldn't test video saving.

### Interesting note:

When sending an image through the web version and taking a screenshot on the phone, the notification did not pop up on Snapchat Web that I took a screenshot, which is what happens when communicating through 2 phones.

### Why It Worked/Didn't Work:

- **Images Success:** The script's hook on `document.createElement` correctly enables right-click saving for images and prevents notifications, as it blocks Snapchat's detection events.
- **Videos Failure:** The black screen issue isn't caused by the script, it's a common Snapchat Web problem.
- **Snapchat Platform Limits:** Web video playback issues have been reported in user forums, unrelated to userscripts like SnapEnhance. Users on reddit report sometimes not even being able to see pictures when using the web version of Snapchat, or the desktop version.

## Test 3 - Read messages privately

**Expected Result:** The script is supposed to prevent read receipts by blocking a web request called `UpdateConversation` that Snapchat sends when you view a message, so the sender doesn't know you've read it.

**Result:** Reading messages wasn't private. When I opened messages on Chrome, my Android phone showed a Bitmoji pop-up, indicating the other person knew I read the message.

### Why It Didn't Work:

- Similar to Typing Issue: The script uses the same `hookPreRequest` function to block `UpdateConversation`, but like Test 1, it might be causing an internal error or being bypassed if Snapchat uses WebSockets for read receipts.
- Bitmoji Pop-Up: The Bitmoji appearance suggests real-time status updates aren't blocked, as the script only logs WebSocket data without modifying it.
- Worker Problems: The injection into background workers could be failing or crashing, similar to the error in Test 1, preventing proper blocking.

### Test 4 - Change active window without losing focus on Snapchat

**Expected Result:** The script is supposed to keep Snapchat thinking the website is always active, even if you switch tabs or windows, by overriding `document.hasFocus` and blocking focus/keydown events.

**Result:** This worked. Throughout the other tests, I could switch tabs on Chrome or click around on my Android phone and recording app, and Snapchat stayed active without pausing.

### Why It Worked:

- Effective Hooks The script's overrides on `document.hasFocus` and event listeners correctly fool Snapchat into detecting constant focus, preventing pauses.
- No Real-Time Conflicts Unlike typing or reading, this feature doesn't rely on network requests or WebSockets, so it avoids the issues seen in Tests 1 and 3.

### Summary

---

So far the only exploits that work are:

- Saving images without notifications
- Keeping Snapchat focused and active when switching tabs

Anything that has to do with altering package data does not work. Features like hiding typing and read receipts rely on intercepting and blocking specific web requests (fetch calls) in Snapchat's background workers. The console error in Test 1 shows the script catches these requests but causes a crash or



error, suggesting Snapchat detects the interference or uses alternative methods like WebSockets.

## Next Steps

---

While features like hiding read receipts and typing receipts are nice, they are not a top priority at the moment. Features that are however necessary if the project is going to remain focused on the web version of Snapchat, are:

- Being able to view videos
- Mask the fact that the web version is being used, as a child would use the app on their phone

## Sources

---

[https://www.reddit.com/r/SnapchatHelp/comments/x9qyt2/why\\_cant\\_i\\_open\\_snaps\\_on\\_snapchat\\_web/](https://www.reddit.com/r/SnapchatHelp/comments/x9qyt2/why_cant_i_open_snaps_on_snapchat_web/)

*SnapEnhance Web*. (n.d.). <https://greasyfork.org/en/scripts/483732-snapenhance-web>